Step 1 - Load & Explore

Phase A · Explicit Schemas · Profiling · ~20 min

> **Goal:** Load all 4 MapleBank CSVs from your Unity Catalog volume with explicit schemas, then profile them like a banking DE: counts, types, nulls, duplicates, and first business questions. Artifact: 01_load_explore.ipynb

LEARN

1. Explicit schemas are non-negotiable in banking. inferSchema turns account number 0123456 into the integer 123456 (leading zero gone — a data-integrity bug auditors will find) and reads the file twice. Always declare a StructType: it is the contract for what the file must look like.

2. import pyspark.sql.functions as F is the convention at every Canadian bank — functions never clash with Python built-ins.

3. Profile before transforming. Always ask first: how many rows, what types, where are the nulls, any duplicates on the business key?

LAB — notebook 01_load_explore (Serverless)

Cell 1 — Setup and schemas:

```
import pyspark.sql.functions as F
from pyspark.sql.types import (StructType, StructField, StringType,
                               DoubleType, DateType, TimestampType)

VOL = "/Volumes/workspace/default/maplebank"

customer_schema = StructType([
    StructField("customer_id",    StringType(), False),
    StructField("first_name",     StringType(), True),
    StructField("last_name",      StringType(), True),
    StructField("sin",            StringType(), True),   # PII - string!
    StructField("date_of_birth",  DateType(),   True),
    StructField("email",          StringType(), True),
    StructField("phone",          StringType(), True),
    StructField("street_address", StringType(), True),
    StructField("city",           StringType(), True),
    StructField("province",       StringType(), True),
    StructField("postal_code",    StringType(), True),
    StructField("customer_since", DateType(),   True),
])

account_schema = StructType([
    StructField("account_id",         StringType(), False),
    StructField("customer_id",        StringType(), False),
    StructField("account_number",     StringType(), True),  # leading zeros!
    StructField("account_type",       StringType(), True),
    StructField("transit_number",     StringType(), True),
    StructField("institution_number", StringType(), True),
    StructField("open_date",          DateType(),   True),
    StructField("balance_cad",        DoubleType(), True),
    StructField("branch_id",          StringType(), True),
])

branch_schema = StructType([
    StructField("branch_id",      StringType(), False),
    StructField("branch_name",    StringType(), True),
    StructField("city",           StringType(), True),
    StructField("province",       StringType(), True),
    StructField("transit_number", StringType(), True),
])

txn_schema = StructType([
    StructField("transaction_id",        StringType(),    False),
    StructField("customer_id",           StringType(),    False),
    StructField("account_id",            StringType(),    False),
    StructField("branch_id",             StringType(),    True),
    StructField("transaction_date",      DateType(),      False),
    StructField("transaction_timestamp", TimestampType(), False),
    StructField("amount_cad",            DoubleType(),    False),
    StructField("transaction_type",      StringType(),    False),
    StructField("merchant_name",         StringType(),    True),
    StructField("channel",               StringType(),    True),
])
```

Cell 2 — Load all four:

```
df_customer = spark.read.option("header", True).schema(customer_schema).csv(f"{VOL}/dim_customer.csv")
df_account  = spark.read.option("header", True).schema(account_schema).csv(f"{VOL}/dim_account.csv")
df_branch   = spark.read.option("header", True).schema(branch_schema).csv(f"{VOL}/dim_branch.csv")
df_txn      = spark.read.option("header", True).schema(txn_schema).csv(f"{VOL}/fact_transactions.csv")

for name, df in [("customers", df_customer), ("accounts", df_account),
                 ("branches", df_branch), ("transactions", df_txn)]:
    print(f"{name:<14} {df.count():>7,} rows   {len(df.columns)} columns")
# Expected: 1,000 / 2,025 / 15 / 10,000
```

Cell 3 — Verify the schema took hold:

```
df_txn.printSchema()   # date=date, amount=double, timestamp=timestamp
df_txn.show(5, truncate=False)
```

Cell 4 — Null profile (the banking habit):

```
def null_profile(df, name):
    print(f"\n-- Null profile: {name} --")
    df.select([
        F.sum(F.when(F.col(c).isNull(), 1).otherwise(0)).alias(c)
        for c in df.columns
    ]).show(vertical=True)

null_profile(df_txn, "transactions")
null_profile(df_customer, "customers")
```

Cell 5 — Duplicate check on business keys:

```
for name, df, key in [("transactions", df_txn, "transaction_id"),
                      ("customers", df_customer, "customer_id"),
                      ("accounts", df_account, "account_id")]:
    total = df.count(); distinct = df.select(key).distinct().count()
    status = "clean" if total == distinct else f"{total-distinct} DUPES!"
    print(f"{name:<14} total={total:>7,} distinct={distinct:>7,}  {status}")
```

Cell 6 — First business questions:

```
df_txn.groupBy("transaction_type") \
      .agg(F.count("*").alias("count"),
           F.round(F.sum("amount_cad"), 2).alias("total_cad")) \
      .orderBy(F.col("total_cad").desc()).show()

big = df_txn.filter(F.col("amount_cad") >= 10000).count()
print(f"Transactions >= $10,000 CAD: {big} ({big/df_txn.count()*100:.1f}%)")
# ~3% large transactions: the seeded FINTRAC-reportable rows
```

COMMIT

File - Export - IPython notebook, upload 01_load_explore.ipynb to pyspark/ on GitHub.

CHECKPOINT

Pass criteria: counts 1,000 / 2,025 / 15 / 10,000; schema shows date/double/timestamp types; zero duplicates on all business keys; ~3% large transactions.